AARHUS UNIVERSITY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Term:	FALL 2022
Examination:	Declarative Programming
Date:	4 January 2023
Duration:	3 hours

REMEMBER to state name and study number on all pages of your submission.

- This is the exam for the lecture Declarative Programming. It consists of 18 questions. You have 3 hours to answer all of them. Strategically, you should first scan the exam paper quickly in its entirety, and identify the groups of questions that seem easiest to you. ***You should answer those first*** and then move on to the others.
- For each question it is indicated how and where the answer must be provided. There are questions that require writing a calculation or the result of a calculation, and questions that require you to write Ocaml or Prolog code and drawings, in particular. Only what is written or drawn inside the specified areas is considered as your answer.
- **If you are in doubt what is expected, please ask!**
- Each question has a weight **[w]**, where **w** is an integer number, specifying the number of points awarded for that question. You can find the weight at the top right corner of each question. The total number of points of all questions together is 100.

Question 1 **[6]**

Recall the signature of the function (=)

```
val (=) : 'a -> 'a -> bool
```

What are the signatures of the functions `e`, `f` and `g`?

```
let k x y = y x
let s x y = x
let u x y z = (x = y) = z
```

Answer:

`val k :` a' -> (a' -> b') -> b'

`val s :` _____

`val u :` _____

Question 2 **[4]**

(a) Define a function with signature `val fapp0 : (int -> int) -> int` that yields the value of its argument when applied to `0`, e.g.,

```
 1 = fapp0 (fun x -> x + 1)
-1 = fapp0 (fun x -> x - 1)
```

Answer: `let`

(b) Define a function with signature `val finc : (int -> int) -> int -> int` that returns a function that yields the value of its first argument when applied to `n+1` (where `n` is the second argument), e.g.,

```
2 = finc (fun x -> x / 2) 3
3 = finc (fun x -> x / 2) 5
3 = finc (fun x -> x - 1) 3
2 = finc (fun x -> x - 2) 3
```

Answer: `let`

Question 3 **[6]**

Complete the following recursive function `revnul xs ys` so that it implements the following behaviour:

```
revnul [] [] = []
revnul [(1,1); (0,2); (3,3); (0,4); (5,5); (0,6) ] [] = [6; 4; 2]
revnul [(1,1); (2,2); (3,3); (4,4) ] [] = []
revnul [(0,1); (0,2); (0,3); (0,4) ] [] = [4; 3; 2; 1]
```

```
let rec revnul xs ys =
  match xs with

  ___________ -> revnul xs' (x::ys)

  ___________ -> revnul xs' ys

  ___________ -> ys
```

Question 4 **[6]**

You are given the following tail recursive implementation of function `isfun f xs ac` that checks wether the list `xs` stores function values of `f` at corresponding indices. E.g.,

```
isfun (fun x -> x * x) [0; 1; 4; 9; 16] 0 = true
isfun (fun x -> x * x) [0; 1; 3; 9] 0 = false
```

```
let rec isfun f xs ac =
  match xs with
    x::xs' -> f ac = x && isfun f xs' (ac + 1)
  | []     -> true
```

Complete the alternative tail recursive function `isfun_alt f xs` below that behaves just as `isfun f xs 0`. You are not allowed to call function `isfun`. Hint: Use the function argument as accumulator.

```
let rec isfun_alt f xs =
  match xs with

    x::xs' -> ____________________________________________

  | []     -> ____________________________________________
```

Question 5 **[4]**

Functions `exists`, `forall`, `neg` and `gt1` are defined as

```
let rec exists f xs =
  match xs with
    [] -> false
  | x::xs' -> f x || exists f xs'

let rec forall f xs =
  match xs with
    [] -> true
  | x::xs' -> f x && forall f xs'

let neg f x = not(f x)

let gt1 x = x >= 1
```

(a) Using functions `forall`, `neg`, and `gt1`, implement function `all_less1` that returns **`true`** if all integer values of a list are smaller than `1`, otherwise **`false`**.
Answer:

```
let all_less1 = ____________________________________________
```

(b) Implement a function `dualall` that behaves exactly like `forall` by means of functions `exists` and `neg` **without** using **`if`**- or **`match`**- expressions. Hint: Think of the logical transformations discussed in the lectures.
Answer:

```
let dualall f = ____________________________________________
```

Question 6 **[7]**

(a) Unnest the following function `one`

```
let one x =
  let two y =
    let bms z = z + y - 1 in
    bms (bms y) - x in
  two (x + x)
```

Answer:

```
let bms ____________________________________________________
let two ____________________________________________________
let one ____________________________________________________
```

(b) Unnest the following function `eeny`

```
let eeny x =
  let meeny y =
    let miny z = x - z * y in
    let moe y = x - y in
    miny (moe x) in
  meeny (x + 1)
```

Answer:

```
let miny ___________________________________________________
let moe ____________________________________________________
let meeny __________________________________________________
let eeny ___________________________________________________
```

Hint: Function declarations can be nested, that is, “inner” functions are declared locally to “outer” functions. Parameters to outer functions can be referred to in inner functions.
Nested functions can be unnested by declaring all “inner” functions globally before the outer function making the passed parameters explicit. The behaviours of the nested and corresponding unnested functions are indistinguishable.

Question 7 **[5]**

Given the type `tree` below

```
type tree = L | N of int * tree * tree
```

and the two functions `rot` and `get`

```
let rec rot t =
  match t with
    N (a, x, N(b, y, z)) -> rot (N (b, N(a, x, y), z))
  | t' -> t'

let rec get t =
  match t with
    N (a, L, _) -> a
  | N (_, x, _) -> get x
  | L -> 0
```

We can compute

```
get (rot (N (1, L, N (2, L, N(3, L, L))))) = 1
```

Reading the function definitions as a system of equations as we have done in the course, calculate the result of the computation by inserting the missing steps:

```
  get (rot (N (1, L, N (2, L, N(3, L, L)))))

= ____________________________________________________

= ____________________________________________________

= ____________________________________________________

= ____________________________________________________

= ____________________________________________________

= 1
```

Question 8 **[5]**

(a) Recall the definition of the functions `foldr` and `concat` given by

```
let rec foldr f a xs =
  match xs with
    [] -> a
  | x::xs' -> f x (foldr f a xs')
```

and

```
let rec concat xs ys =
  match xs with
    [] -> ys
  | x::xs' -> x::concat xs' ys
```

Using these function implement a function `flatten` that "flattens" nested lists as follows

```
[] = flatten []
[1; 2; 3; 4] = flatten [[1; 2]; [3; 4]]
[1; 1; 2; 3; 3; 4] = flatten [[1]; [1; 2; 3]; [3; 4]]
```

Answer:

`let flatten =` _____

Note: You are not allowed to add parameters before the `=`.

(b) Using function `compose` in addition where

```
let compose f g x = f (g x)
```

implement a function `flatsum` that adds all integer values of a list of lists, that is,

```
0 = flatsum [[]]
0 = flatsum [[]; []]
1 = flatsum [[]; [1]]
3 = flatsum [[1]; [2]]
5 = flatsum [[2]; [1; 2]]
21 = flatsum [[1; 2; 3]; [4]; [5; 6]]
```

Answer:

`let flatsum =` _____

Note: You are not allowed to add parameters before the `=`.

Question 9 **[8]**

The following function `tp` computes the transpose of a list of lists.

```
let rec maphd xs =
  match xs with
    [] -> []
  | []::zs -> maphd zs
  | (y::_)::zs -> y::maphd zs

let rec maptl xs =
  match xs with
    [] -> []
  | []::zs -> maptl zs
  | (_::ys)::zs -> ys::maptl zs

let rec tp xs =
  match xs with
    []::_ -> []
  | _ -> maphd xs::tp (maptl xs)
```

Considering the function definitions as a system of equations, calculate the missing steps leading from `tp [[1; 2]; [3; 4]]`
to `[1; 3]::maphd [[2]; [4]]::tp (maptl [[2]; [4]])`.

```
  tp [[1; 2]; [3; 4]]
= ____________________________________________________
= ____________________________________________________
= ____________________________________________________
= ____________________________________________________
= ____________________________________________________
= ____________________________________________________
= ____________________________________________________
= ____________________________________________________
= [1; 3]::maphd [[2]; [4]]::tp (maptl [[2]; [4]])
```

Question 10 **[6]**

Consider the pattern matching algorithm match from the course

$$
\begin{array}{lll}
\mathrm{match}\ S\ u\ =\ u & & \text{if } S = \emptyset \\
\qquad\qquad \mathrm{match}\ (S \setminus (x,t))\ u & & \text{if } (x,t) \in S \cap u \\
\qquad\qquad \mathrm{match}\ (S \setminus (x,t))\ (u \cup \{(x,t)\}) & & \text{if } (x,t) \in S \wedge (x,s) \notin u \text{ for all } s \\
\qquad\qquad \mathrm{match}\ (S \cup \{(s_1,s'_1),\ldots,(s_k,s'_k)\}\ u & & \text{if } (c(s_1,...,s_k), c'(s'_1,...,s'_k)) \in S \wedge c = c' \\
\qquad\qquad \mathbf{fail} & & \text{otherwise}
\end{array}
$$

Let m, n, w, x and y denote variables and F, G, H, P, Q, S and Z constants. As usual, we leave away empty argument lists, e.g., instead of $Z()$ we write Z.

Calculate:

(a) The value of $\mathrm{match}\ \{(H(m,n,G(m,n)), H(P,G(w),G(P,Q))\}\ \emptyset$

$\mathrm{match}\ \{(H(m,n,G(m,n)), H(P,G(w),G(P,Q)))\}\ \emptyset$

$=$ _____

$=$ _____

$=$ _____

$=$ _____

$=$ _____

$=$ _____

(b) The value of $\mathrm{match}\ \{(F(S(x),y), F(S(Z),S(Z)))\}\ \emptyset$

$\mathrm{match}\ \{(F(S(x),y), F(S(Z),S(Z)))\}\ \emptyset$

$=$ _____

$=$ _____

$=$ _____

$=$ _____

$=$ _____

Question 11 **[5]**

Using the truth table method, determine whether the formula

$$\neg(P \Rightarrow Q) \Rightarrow (\neg Q \Rightarrow P)$$

is satisfiable.

Fill in the truth table:

P	Q	$\neg(P \Rightarrow Q) \Rightarrow (\neg Q \Rightarrow P)$

Tick the correct answer:

☐ The formula is satisfiable.

☐ The formula is unsatisfiable.

Question 12 **[4]**

Determine whether the formula

$$\begin{array}{ll} & H \vee \neg U \vee L \\ \wedge & \neg G \vee \neg H \vee \neg L \\ \wedge & G \vee H \vee \neg L \\ \wedge & \neg G \vee H \vee \neg L \\ \wedge & \neg H \end{array}$$

is satisfiable applying the DPLL algorithm, using the rules

(UR) the 1-literal rule (aka unit clause rule),
(PR) the affirmative-negative rule (aka pure literal rule),
(SR) the splitting rule.

Fill into the boxes the formulas resulting from the rule applications indicated between the boxes.

1:
$$\boxed{\begin{array}{ll} & H \vee \neg U \vee L \\ \wedge & \neg G \vee \neg H \vee \neg L \\ \wedge & G \vee H \vee \neg L \\ \wedge & \neg G \vee H \vee \neg L \\ \wedge & \neg H \end{array}}$$

Applying rule **(UR)** to the formula in box 1 yields:

2: $\boxed{}$

Applying rule **(PR)** to the formula in box 2 yields:

3: $\boxed{}$

Applying rule **(PR)** to the formula in box 3 yields:

4: $\boxed{}$

Tick the correct answer:
☐ The formula is satisfiable.
☐ The formula is unsatisfiable.

Question 13 **[6]**

The following table specifies two lambda terms in the first row. Complete the empty cells in the remaining rows of the table according to the lambda terms for each column.
Remark: In your answers you may write "L" to represent "λ" if you prefer.

Lambda term	$(\lambda f \cdot f\ 5)(\lambda g \cdot g + z)$	$\lambda w\ q \cdot (q\ (\lambda v \cdot v\ v))$
Name the **type** of lambda term		
Rewrite the term in its fully curried form		
Specify the **free** variables		
Specify the **bound** variables		

Question 14 **[6]**

Consider the following Prolog program:

```
animal(moa).
animal(weta).
animal(huhu_grub).
animal(tuatara).
animal(kiwi).
animal(eagle).
plant(totara).

eats(tuatara,   weta).
eats(huhu_grub, totara).
eats(eagle,     tuatara).
eats(kiwi,      huhu_grub).
eats(eagle,     kiwi).
eats(kiwi,      totara).
eats(tuatara,   huhu_grub).

predator(X) :- animal(X), eats(X, Y), animal(Y).

apex_predator(X) :- predator(X), not(eats(_,X)).
```

(a) For each of the following Prolog queries applied to the above program, write down the first query result when executed in Prolog. For example:

```
?- animal(X).
```

First query result when executed in Prolog:

```
X = moa.
```

(i) `?- eats(X,huhu_grub).`

First query result when executed in Prolog:

_____

(ii) `?- eats(X,Y).`

First query result when executed in Prolog:

_____

(iii) `?- apex_predator(X).`

First query result when executed in Prolog:

_____

(b) A *herbivore* is an animal that only eats plants. Write this definition of the *herbivore* predicate as a Prolog rule:

```
herbivore(X) :- ____________________________________________
```

(c) An organism X is said to be *higher up in the food chain* than organism Z if X eats Z, or there exists n organisms $Y_1, Y_2, ..., Y_n$ such that $eats(X, Y_1)$, $eats(Y_1, Y_2)$, $\ldots$, $eats(Y_n, Z)$, for $1 \leq n$. Define the predicate *organism* and the binary relation *higher_up* with Prolog rules using *recursion*.

Hint: Recall that ";" is disjunction in Prolog.
Hint: for the *higher_up* relation you'll need to define a base case rule and a recursive case rule.

```
organism(X) :- ________________________________________
_____________________________________________________
higher_up(X,Z) :- ____________________________________
_____________________________________________________
higher_up(X,Z) :- ____________________________________
_____________________________________________________
_____________________________________________________
_____________________________________________________
```

Question 15 **[5]**

(a) Write down a **variant** of the following Prolog fact and rule:

- `m(X, x(1), miro(M)).`

  Answer:

- `w(X,W) :- likes(Likes,X), likes(Likes,W).`

  Answer:

(b) Write down an **instance** of the following Prolog terms:

- `sphere(centre(X,Y,Z), radius(R))`

  Answer:

- `t(q(T), w(X,t(T)), W, x(X, W))`

  Answer:

(c) Given the following Prolog term and substitution, perform the substitution:

- `del(qT(Rd), rD, Qt) {Qt=rD(del), Rd=T}`

  Answer:

Question 16 **[8]**

(a) Write the following as Prolog rules and facts in a Prolog program:

- all greek gods are immortal
- Zeus, Gaia and Apollo are greek gods
- Zeus is older than Apollo
- Gaia is older than every greek god

Hint: You will need to implement predicates *immortal*, *greek_god*, and *older*. Consider how many arguments each of these predicates needs.

Answer:

(b) Use your program to write the following questions as Prolog queries in the Prolog terminal, and to give Prolog's answer to each query.

(i) Is Gaia immortal?

Prolog query:

```
?-
```

First query result when executed in Prolog:

(ii) Who is younger than the greek god that is younger than Gaia?

Prolog query:

```
?-
```

First query result when executed in Prolog:

(iii) Is Gaia younger than herself?

Prolog query:

```
?-
```

First query result when executed in Prolog:

_____

_____

Question 17 **[4]**

In the lecture we discussed the unification procedure, consisting of six rewriting "rules" that step-by-step transform a set of unification expressions until arriving at a "solved substitution" or a failure.

The six rules are: *swap*, *eliminate*, *decompose*, *conflict*, *occurs*, *delete*.

Apply the same procedure to unify the expressions in (a) and (b) below. Apply exactly one "rule" at each step, and specify the rule used by writing the rule name to the right side of the new unification expression set (as we did in lectures).
Remark: In your answers you can write "$=$" to represent "$\approx$" if your prefer.

(a) Unify the term `q(w(c,x(X)),Z, Y)` with the term `q(w(G,Y),c, x(G))`.

Answer:

$\{q(w(c, x(X)), Z, Y) \approx q(w(G, Y), c, x(G))\}$

$\rightarrow$ _____

_____

_____

_____

_____

_____

_____

_____

(b) Unify the term `r(r(r(A), B), A, C)` with the term `r(r(C, r(D)), B, r(D))`.

Answer:

$\{r(r(r(A), B), A, C) \approx r(r(C, r(D)), B, r(D))\}$

$\rightarrow$ _____

_____

_____

_____

_____

_____

_____

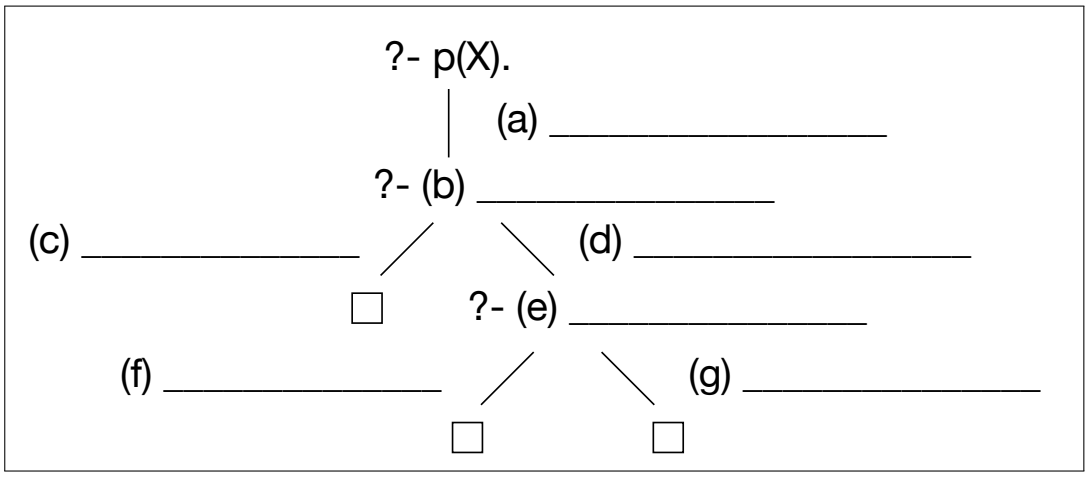
_____

Question 18 **[5]**

Consider the following Prolog rules that define the predicate `q/2`:

```
q(q).
q(r).
r(q,r).
r(X,X) :- q(X).
p(X) :- r(X,_).
```

Consider the following Prolog query:

```
p(X)
```

Complete the missing goals and variants at the labels (a) to (f) for this query in the following search tree (SLD-tree)